

Java

Лекція 2

Зміст

- `final`: змінні, методи, класи
- `Switch expression`
- `Enum`
- `Record`
- `Equals, hashCode`
- Винятки: `throw, try/catch`, валідація даних
- `abstract`: класи без екземплярів

final: три застосування

```
public static final int MAX_YEAR_OFFSET = 1;    // 1) константа: не змінити

public final void setYear(int year) { ... }      // 2) метод: не перевизначити

public final class GroupUtils { ... }           // 3) клас: не наслідувати
```

- `static final` + ВЕЛИКІ_ЛІТЕРИ — так оформлюються константи
- `private final String name;` // поле екземпляра: присвоїти один раз, у конструкторі
- `final`-метод — «цю поведінку нащадкам змінювати заборонено». Часто методи класу, які викликають у конструкторах, роблять `final`, щоб конструктор не міг випадково викликати перевизначену версію з нащадка — його поля на цей момент ще не ініціалізовані
- `final`-клас — «від цього класу не наслідуються» (`String` — саме такий!)

Що станеться без final

Code

```
class Car {  
    protected int year;
```

1

```
    Car(int year) {  
        setYear(year);  
    }
```

final

```
    public void setYear(int year) {  
        this.year = year;  
    }  
}
```

3

```
class TaxiCar extends Car {  
    private String city = "Kyiv";
```

```
    TaxiCar(int year) { super(year); }
```

```
    @Override
```

```
    public void setYear(int year) {  
        System.out.println(city.toUpperCase()); // city ==  
        null !  
        this.year = year;  
    }  
}
```

2

```
TaxiCar taxi = new TaxiCar(2020);
```

Switch-case

```
int day = 3;
String dayName;

switch(day) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday";
        break;
    default:
        dayName = "Other";
}

System.out.println(dayName);
```

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday: Start work week!");
        break;
    case 2:
        System.out.println("Tuesday: Team meeting");
        break;
    case 3:
        System.out.println("Wednesday: Work from home");
        break;
    case 4:
        System.out.println("Thursday: Project review");
        break;
    case 5:
        System.out.println("Friday: Wrap up tasks");
        break;
    case 6:
    case 7:
        System.out.println("Weekend: Relax!");
        break;
    default:
        System.out.println("Invalid day");
}
```

Недоліки `switch-case`

У старому `switch` для присвоєння значень потрібно оголошувати змінну заздалегідь

Необхідні `break` → легко забути → `fall-through`.

Код стає громіздким, особливо коли потрібно багато кейсів.

```
String dayName;
```

```
switch(day) {
```

```
    case 1: dayName = "Monday"; break;
```

```
    case 2: dayName = "Tuesday"; break;
```

```
    default: dayName = "Other";
```

```
}
```

Switch expression

Використовується як вираз (*expression*), а не лише як оператор.

Менше *boilerplate* → немає *break*, компактно і безпечно.

Кожен *case* повертає значення

```
int day = 3;
```

```
String dayName = switch(day) {
```

```
    case 1 -> "Monday";
```

```
    case 2 -> "Tuesday";
```

```
    case 3 -> "Wednesday";
```

```
    default -> "Other";
```

```
};
```

```
System.out.println(dayName);
```

Switch expression

Якщо потрібно виконати кілька дій, використовується блок у фігурних дужках. Якщо треба повернути значення з блока, то використовуємо `yield`.

```
int day = 3;

String message = switch (day) {
    case 1 -> {
        System.out.println("Monday: Start work week!");
        yield "Monday";
    }
    case 2 -> {
        System.out.println("Tuesday: Team meeting");
        yield "Tuesday";
    }
    case 3 -> {
        System.out.println("Wednesday: Work from home");
        yield "Wednesday";
    }
}
```

```
    }
    case 4 -> {
        System.out.println("Thursday: Project review");
        yield "Thursday";
    }
    case 5 -> {
        System.out.println("Friday: Wrap up tasks");
        yield "Friday";
    }
    case 6, 7 -> {
        System.out.println("Weekend: Relax!");
        yield "Weekend";
    }
    default -> {
        System.out.println("Invalid day");
        yield "Invalid";
    }
};

System.out.println("Returned value: " + message);
```

Switch-case vs Switch expression

Старий синтаксис (case ... :)

✓ Використовувати, коли:

Потрібна сумісність із кодовою базою до Java 14.

Свідомо потрібен fall-through — кілька міток мають продовжувати одна одну.

✗ Недоліки:

Потребує break, інакше — випадковий fall-through.

Не можна використати як вираз (для присвоєння чи return).

Змінні, оголошені в одному case, видно в наступних — спільна область видимості.

```
case STOP -> service.stop();

default -> log.warn("Невідома команда");

}
```

// як вираз — повертає значення

```
String dayName = switch (day) {

    case 1 -> "Monday";

    default -> "Other";

};
```

Switch expression (з Java 14+)

✓ Використовувати, коли:

Практично завжди, коли не потрібен fall-through. Працює в обох ролях:

// як оператор — виконує дії

```
switch (command) {

    case START -> service.start();
```

✓ Переваги:

Немає break і випадкового fall-through.

Кожна гілка має власну область видимості.

Кілька міток через кому: case 6, 7 ->

У формі виразу компілятор вимагає повноти — забутий варіант enum не пройде компіляцію.

Pattern Matching y switch (Java 17+)

```
static String format(Object obj) {
```

```
    if (obj instanceof String) {
```

```
        String s = (String) obj;
```

```
        return "String of length " + s.length();
```

```
    } if (obj instanceof Integer) {
```

```
        Integer i = (Integer) obj;
```

```
        return "Integer value " + i;
```

```
    }
```

```
    return "Unknown type";
```

```
}
```

```
static String format(Object obj) {
```

```
    return switch (obj) {
```

```
        case String s -> "String of length " + s.length();
```

```
        case Integer i -> "Integer value " + i;
```

```
        case null -> "Null value";
```

```
        default -> "Unknown type";
```

```
    };
```

```
}
```

var та text blocks

◆ var (Java 10)

Використовується для локальних змінних.

Тип визначається автоматично компілятором.

Зменшує "шум" у коді, але не робить Java динамічною.

```
var name = "Alice";    // String
var age = 25;          // int
var list = new ArrayList<String>();
```

✓ Зручно, коли тип і так зрозумілий з правої частини.

✗ Не можна для полів класів чи параметрів методів.

◆ Багаторядковий текст (Text Blocks, Java 15)

Огортається в потрібні лапки ""

Автоматично зберігає форматування.

Зручно для SQL, HTML, JSON тощо.

```
String json = ""
{
    "name": "Alice",
    "age": 25
}
"";
```

✓ Краще читається, ніж конкатенація "..." + "...".

✓ Підтримка інтерполяції через `String::formatted` (Java 15).

"Привіт, " + name

Форматування рядків

Класичне `String.format` (Java 1.5+)

Статичний метод, який приймає шаблон і аргументи:

```
String.format("Привіт, %s! Тобі %d років.", name, age)
```

`String::formatted` (Java 15+)

Той самий форматувальник, але як метод екземпляра — шаблон стає отримувачем виклику:

```
"Привіт, %s! Тобі %d років.".formatted(name, age)
```

З'явився разом із текстовими блоками і найкраще працює саме з ними: спершу читається сам текст, а вже потім — чим його заповнюють.

```
"""
```

```
Привіт, %s!
```

```
Тобі %d років.
```

```
"".formatted(name, age)
```

Різниця суто синтаксична. Обидва варіанти не типобезпечні: невідповідність `%d` і переданого аргумента виявиться тільки під час виконання, кинувши `IllegalFormatConversionException`.

Enum

- Enum (enumeration) — спеціальний різновид класу, який задає фіксовану множину іменованих констант. Неявно успадковує `java.lang.Enum`, тому не може мати іншого батьківського класу.
- Використовується для кращої читабельності та безпеки коду.
- Замість чисел або рядків
- Enum може містити:
 - Поля
 - Конструктори
 - Методи
- Може використовуватися у `switch` як значення, що перевіряється

Enum. Приклад

```
enum Day {
    MONDAY("Початок тижня"),
    TUESDAY("Другий день тижня"),
    WEDNESDAY("Середина тижня"),
    THURSDAY("Четвер"),
    FRIDAY("Кінець робочого тижня"),
    SATURDAY("Вихідний"),
    SUNDAY("Вихідний");

    // Поле для опису
    private final String description;

    // Конструктор Enum
    Day(String description) {
        this.description = description;
    }

    // Метод для отримання опису
    public String getDescription() {
        return description;
    }
}
```

```
public class EnumDemo {
    public static void main(String[] args) {
        Day today = Day.WEDNESDAY;

        System.out.println("Сьогодні: " + today);
        System.out.println("Опис: " + today.getDescription());

        System.out.println("\nУсі дні тижня:");
        for (Day d : Day.values()) {
            System.out.println(d + " - " + d.getDescription());
        }

        switch (today) {
            case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY ->
                System.out.println("\nРобочий день");
            case SATURDAY, SUNDAY ->
                System.out.println("\nВихідний день");
        }

        Day d = Day.valueOf("FRIDAY");
        System.out.println("\nМетод valueOf: " + d);
        System.out.println("Позиція у Enum (ordinal): " + d.ordinal());
    }
}
```

Аналогічний клас до Enum

```
public class DayClass {  
    // Статичні поля (імітація констант Enum)  
    public static final DayClass MONDAY = new DayClass("MONDAY", "Початок  
тижня");  
    public static final DayClass TUESDAY = new DayClass("TUESDAY", "Другий  
день тижня");  
    public static final DayClass WEDNESDAY = new DayClass("WEDNESDAY",  
"Середина тижня");  
    public static final DayClass THURSDAY = new DayClass("THURSDAY",  
"Четвер");  
    public static final DayClass FRIDAY = new DayClass("FRIDAY", "Кінець  
робочого тижня");  
    public static final DayClass SATURDAY = new DayClass("SATURDAY",  
"Вихідний");  
    public static final DayClass SUNDAY = new DayClass("SUNDAY", "Вихідний");  
  
    // Поля для назви та опису  
    private final String name;  
    private final String description;  
  
    // Приватний конструктор  
    private DayClass(String name, String description) {  
        this.name = name;  
        this.description = description;  
    }  
}
```

```
}  
  
// Геттери  
public String getName() {  
    return name;  
}  
  
public String getDescription() {  
    return description;  
}  
  
// Метод для зручного виводу  
@Override  
public String toString() {  
    return name + " (" + description + ")";  
}  
  
// Масив всіх значень (імітація values())  
public static DayClass[] values() {  
    return new DayClass[]{MONDAY, TUESDAY, WEDNESDAY, THURSDAY,  
FRIDAY, SATURDAY, SUNDAY};  
}  
}
```

Переваги використання `Enum`

- Зрозумілість коду
- Безпечність типів
- Легко розширювати (поля, методи)
- Підходить для `switch` та колекцій

Record (Java 16+)

- Це спеціальний клас у Java (Java 16+), який використовується для зберігання даних.
- Основна ідея: менше шаблонного коду для класів, що містять лише поля та геттери.
- Значення, перелічені в дужках, називаються компонентами `record`.
- `Record` автоматично генерує:
 - Поле `private final` для кожного компонента (значення не можна змінювати після створення об'єкта)
 - Метод-`accessor` з іменем компонента: `name()`, а не `getName()`
 - Конструктор з усіма компонентами (канонічний конструктор)
 - Методи `equals()`, `hashCode()`, `toString()`
 - Може містити свої методи, а також `static` поля — але не додаткові нестатичні поля: увесь стан описують лише компоненти.

Конструктори у Record

У record є три види конструкторів.

Канонічний (canonical) — той, чия сигнатура точно збігається зі списком компонентів. Він існує завжди: якщо не написати жодного, компілятор згенерує його сам і просто присвоїть поля. Канонічний конструктор не може бути менш видимим за сам record (у публічному record він мусить бути public).

```
public record Point(int x, int y) { }  
// компілятор генерує: public Point(int x, int y) { this.x = x; this.y = y; }  
Його можна написати явно:  
public record Point(int x, int y) {  
    public Point(int x, int y) {  
        if (x < 0 || y < 0) throw new IllegalArgumentException("Від'ємні  
координати");  
        this.x = x;  
        this.y = y;  
    }  
}
```

Компактний (compact) — скорочена форма канонічного: без списку параметрів і без дужок. Параметри доступні за іменами компонентів, а присвоєння `this.x = x` компілятор дописує сам у кінці тіла.

```
public record Point(int x, int y) {  
    public Point {  
        if (x < 0 || y < 0) throw new IllegalArgumentException("Від'ємні  
координати");  
    }  
}
```

Це основна форма для валідації та нормалізації. Параметрам можна присвоювати нові значення — присвоїться саме змінена версія:

```
public record Person(String name, List<String> phones) {  
    public Person {  
        name = name.trim();  
        phones = List.copyOf(phones);  
    }  
}
```

⚠ Важливо: у компактному конструкторі не можна писати `this.name = ...` — тільки працювати з параметрами.

Додаткові (alternative) — звичайні перевантажені конструктори. Кожен з них зобов'язаний першим рядком делегувати через `this(...)` до о іншого конструктора — ланцюжок завжди завершується канонічним, бо тільки він присвоює полям значення.

```
public record Point(int x, int y) {  
    public Point(int x) {  
        this(x, 0);  
    }  
    public Point() {  
        this(0, 0);  
    }  
}
```

Record. Приклад

```
public record Person(String surname, String name,
    LocalDate birthDate) {

    public Person {
        if (surname == null || surname.isBlank()) {
            throw new IllegalArgumentException("Surname
cannot be empty");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name
cannot be empty");
        }
        if (birthDate == null ||
birthDate.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("Birth date
is invalid");
        }
    }
}
```

```
public Person(String surname, String name) {
    this(surname, name, LocalDate.now());
}

public boolean isAdult() {
    return Period.between(birthDate,
LocalDate.now()).getYears() >= 18;
}
}
```

```
Person p = new Person("Іваненко", "Олександр",
    LocalDate.of(2005, 5, 15));
System.out.println(p); //
Person[surname=Іваненко, name=Олександр,
    birthDate=2005-05-15]
System.out.println("Повнолітній? " +
    (p.isAdult() ? "Так" : "Ні"));
```

Переваги та обмеження Record

✓ Переваги

- Менше шаблонного коду (автоматично генерує конструктор, геттери, equals, hashCode, toString)
- Immutable (поля final) → безпечний для багатопоточності
- Ідеально підходить для DTO / value objects
- Чітка структура даних і типобезпека

⚠ Обмеження

- Поля final → значення не можна змінювати після створення (але це shallow immutability: вміст List чи масиву захищений не буде)
- Record не може успадковувати клас і сам неявно final — від нього не успадковуються класи
- Не можна створити об'єкт частково — усі компоненти обов'язково отримують значення». Але може бути record взагалі без компонентів - record Empty() { }

Object

Метод	Призначення	Приклади використання / перевизначення
<code>toString()</code>	Повертає рядкове представлення об'єкта	Для зрозумілого виводу, логування
<code>equals(Object obj)</code>	Перевірка рівності об'єктів	Для порівняння об'єктів за значенням, а не посиланням
<code>hashCode()</code>	Повертає числовий хеш об'єкта	Разом з <code>equals()</code> для колекцій HashMap, HashSet
<code>clone()</code>	Створює копію об'єкта	Іноді для створення копій об'єктів (часто замінюють <code>copy constructor</code>)
<code>getClass()</code>	Повертає Class об'єкта	Використовується для reflection, порівняння типів

Правила для equals та hashCode

1 Правила для equals()

- Симетричність: `a.equals(b)` має давати той самий результат, що і `b.equals(a)`.
- Рефлексивність: `a.equals(a)` завжди має бути `true`.
- Транзитивність: якщо `a.equals(b)` і `b.equals(c)` $\rightarrow$ `a.equals(c)` теж має бути `true`.
- Консистентність: багаторазові виклики `a.equals(b)` без зміни об'єктів дають один і той самий результат.
- null-check: `a.equals(null)` має повертати `false`.
- Використовувати ті поля, які визначають "логічну рівність":
 - Краще `final` поля, які рідко змінюються

- Не включати колекції, які часто змінюються, бо це може порушити консистентність

2 Правила для hashCode()

- Якщо `a.equals(b)` $\rightarrow$ `hashCode(a) == hashCode(b)`.
- Якщо `a.equals(b)` повертає `false` $\rightarrow$ hash-коди можуть бути однакові, але бажано різні для розподілу у хеш-структурах.
- Використовувати ті ж поля, що й у `equals()`.
- Не включати `mutable` поля, які часто змінюються, щоб `hashCode` не змінювався після додавання в колекції типу `HashMap` або `HashSet`.

Приклад

```
public class Person {  
    private final String surname;  
    private final String name;  
    private final LocalDate birthDate;  
    private List<String> courses;  
  
    public Person(String surname, String name, LocalDate birthDate,  
List<String> courses) {  
        this.surname = Objects.requireNonNull(surname);  
        this.name = Objects.requireNonNull(name);  
        this.birthDate = Objects.requireNonNull(birthDate);  
        this.courses = courses; // змінюване поле  
    }  
  
    public String getSurname() { return surname; }  
    public String getName() { return name; }  
    public LocalDate getBirthDate() { return birthDate; }  
    public List<String> getCourses() { return courses; }  
  
    @Override
```

```
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Person person)) return false;  
        return surname.equals(person.surname) &&  
            name.equals(person.name) &&  
            birthDate.equals(person.birthDate);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(surname, name, birthDate);  
    }  
  
    @Override  
    public String toString() {  
        return surname + " " + name + ", народився: " + birthDate +  
            ", курси: " + courses;  
    }  
}
```

throw та try/catch: сигнал «так не можна» і реакція на нього

- throw перериває виконання методу — некоректне значення не буде присвоєне
- IllegalArgumentException — стандартний виняток «поганий аргумент»
- Повідомлення пишемо корисним: що очікувалось і що прийшло
- Код у try виконується до першого винятку
- catch перехоплює його: програма не падає, а реагує
- e.getMessage() — те повідомлення, що ми поклали у throw

```
public void setYear(int year) {  
    if (year < 1886) {    // перший автомобіль :)  
        throw new IllegalArgumentException(  
            "year must be >= 1886, got: " + year);  
    }  
    this.year = year;  
}
```

```
try {  
    Car old = new Car("Benz", 1700);  
    System.out.println("created");    // сюди не дійдемо  
} catch (IllegalArgumentException e) {  
    System.out.println("rejected: " + e.getMessage());  
}
```

Абстрактний клас (екземпляр НЕ МОЖНА СТВОРИТИ)

- `abstract` = «екземплярів не буває, тільки нащадки»
- Використовуємо, коли базовий клас — спільний код, а не самотійна сутність («продукт взагалі» не існує — існує електроніка, їжа...)

```
public abstract class Product {  
    protected String name;  
    protected double price;  
  
    protected Product(String name, double price) { ... }  
}  
  
Product p = new Product("?", 0);    // ПОМИЛКА КОМПІЛЯЦІЇ  
Product p = new Electronics(...);  // ок: створюємо нащадка
```

Абстрактні методи

- Методи без реалізації.
Реалізація буде в нащадках:

```
public abstract double  
calculateFinalPrice();
```

- Якщо хоча б один метод в класі абстрактний, то клас повинен бути оголошений теж як **abstract**

```
public abstract class Product {  
    protected String name;  
    protected double price;  
  
    protected Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public abstract double calculateFinalPrice();  
}  
  
public class Electronics extends Product {  
    public Electronics(String name, double price){ super(name, price); }  
  
    @Override  
    public double calculateFinalPrice() { return price * 1.10; }  
}  
  
public class Food extends Product {  
    public Food(String name, double price) { super(name, price); }  
  
    @Override  
    public double calculateFinalPrice() { return price * 0.80; }  
}
```

Підсумок

- `var` – спрощене оголошення локальних змінних
- ◆ `Switch expression` – компактний код із можливістю повертати значення
- `Enum` – безпечні перелічувані константи з полями та методами
- `Record - immutable` клас для даних із автоматичними `constructor`, `equals()`, `hashCode()`, `toString()`
- ⚡ `equals()` & `hashCode()` – перевизначаємо для коректної роботи колекцій, враховуючи стабільні поля
- ✎ `toString()` – зрозуміле рядкове представлення об'єктів
- 🏗️ `abstract` – клас-каркас, який не можна інстанціювати; з'ясували, що абстрактний метод у ньому не обов'язковий
- 🔒 `final` – незмінні змінні, методи без перевизначення та класи, закриті для успадкування